AMBF Raven2 Project Onboarding Document

Sean Fabrega | Mount Holyoke College IMeRo Lab Clapp 222

Professor Melody Su | Summer 2021

Table of Contents:

Table of Contents:

Introduction
- Goal of the Project
- Raven II and Tele-operated Minimally Invasive Surgery
- Autonomous Multibody Framework (AMBF)

Setting Up the Controller

Pieces of the Controller
- ambf_raven_def.py
- ambf_raven.py
  - Variables
  - Methods
    - go_home
    - sine_dance
    - manual_move
    - move
    - control_move
- raven_fk.py
- raven_ik.py
- ambf_raven_controller.py
- utilities.py

Capabilities of the Controller
- Homing
- Sine Dance
- Manual Move

Limitations of the Controller
- Slow to move short distances

No manual controller in real time
Breaks when it makes contact with soft bodies

Next steps

Introduction

The field of intelligent robotic surgery promises great technological strides in the performing of minimally invasive surgery, most notably increasing access and decreasing costs for these procedures. However, research to improve the accuracy of intelligent surgical robots remains expensive and difficult to undergo on a large scale. This presents a barrier to developing larger datasets for training machine learning models to increase the efficiency and accuracy of robot performance. The goal of this project is to use simulated robots to develop this dataset and ultimately use it to train a machine learning model which could transform vision data to force (haptic feedback) data. This would pioneer a new, cheaper alternative for creating these datasets in intelligent medical robotic research and for generating real-time haptic feedback using a software solution as opposed to hardware sensors, which are expensive.

Goal of the Project

My specific piece of the process outlined above is to develop a client for controlling a rendering of the Raven II Surgical Robot in the Autonomous Multibody Framework developed by Adnan Munawar. Prior to my work on this project, there was a C++ client developed by Melody Su which had similar capabilities to my own, however it was in C++ and was not compatible with many other capabilities within the AMBF. Much of my client translates code from the C++ client (which is largely developed from the Raven II base code) and extends certain capabilities, such as manual control. Eventually, I intend to use this client to create a dataset of vision and haptic feedback in order to train the model I defined above.

Raven II and Tele-operated Minimally Invasive Surgery

The Raven II is specifically designed for research in tele-operated medical robots, meaning it is not intended for conducting human operation, but rather exploring the field in an academic setting. It has two identical arms, seven joints per arm, and is cable driven. This means that each joint is controlled by thin, metal cables stretching from the joint to a motor in the base of the robot. It is similar to the way we move our fingers: using tendons that connect to muscles

in our arms. On the end of each arm is a gripper. There are multiple attachments for the end of the arms, however for my research, I assumed that the robot would always use a gripper.
The Raven II models a tele-operated minimally invasive surgical robot, meaning it is intended for studying minimally invasive surgery where the surgeon and the patient do not need to be in the same room. Instead, the robot is conducting the surgery while the surgeon is controlling the arms using their hands in real time and viewing a screen of what is happening inside the patient. Haptic feedback is used to simulate the force acting upon the surgeon when they conduct the surgery, meaning the more accurate the haptic feedback data, the more precise and accurate a surgeon can be during an operation. If the surgeon is able to conduct surgery remotely, this opens up the capacity for surgery to be conducted in hard to reach places, such as in rural or geographically isolated areas, or in dangerous environments for the surgery, such as battlefields. Typically, sensors on the tip of the grippers are used to generate this information, however, these sensors either have to be disposable or sturdy enough to undergo the process of sterilization (which usually involves extreme heat). Either way, the sensors tend to be very expensive and therefore provide a cost barrier to undergoing this type of tele-operated surgery. Therefore developing a software solution would decrease cost and increase the capability for these tele-operated operations to be conducted.

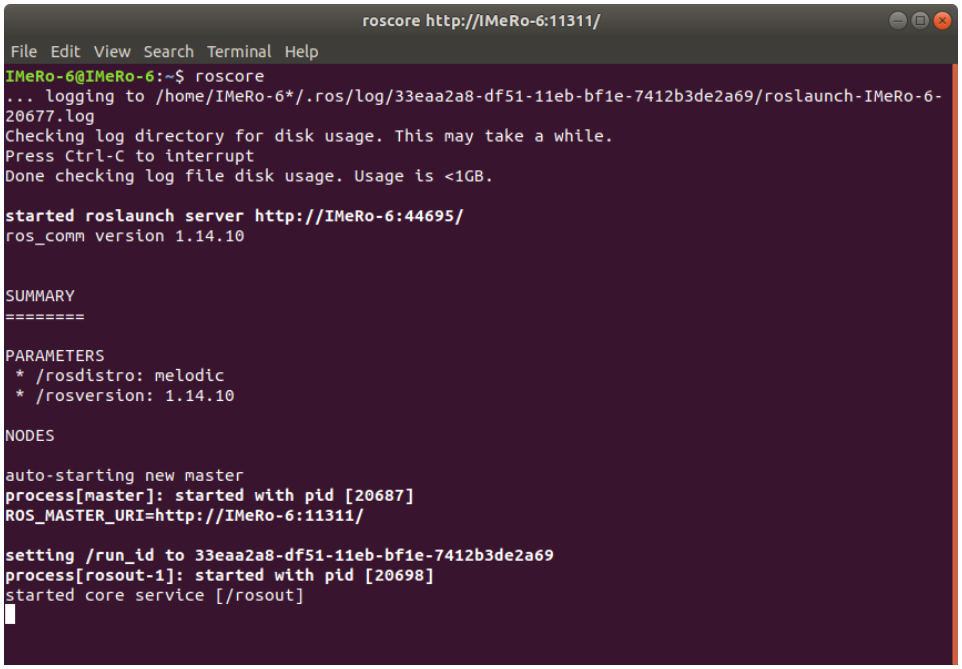
Autonomous Multibody Framework (AMBF)

Another concern when developing this model is the cost of collecting data. In order to create an accurate model, one needs a large dataset specifically looking at the interaction between the robot grippers and soft body tissue. There are few materials that mimic the force exerted by human tissue and the acquiring and study of research medical robots are also quite expensive. If an open-source simulator were to successfully create the same dataset, that would significantly decrease the cost of development and increase the capabilities for a larger scale dataset that isn't constrained by the reality of using soft body tissue in a lab.
The Autonomous Multibody Framework (AMBF) provides a solution by simulating the Raven II in 3D space while considering gravity and other objects in the environment.

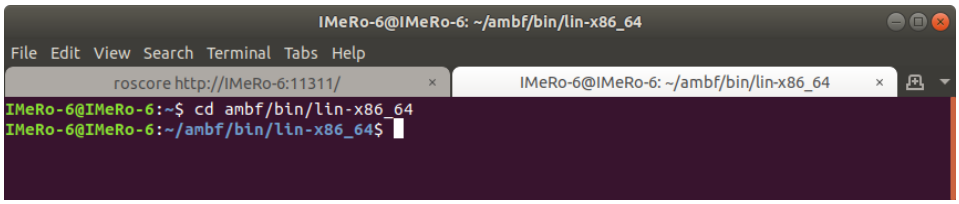
Setting Up the Controller

1. Install ROS on your computer
2. Clone this repository onto your computer
3. Follow these instructions for building ambf
4. Once you've installed ROS and build ambf on your computer, open your terminal

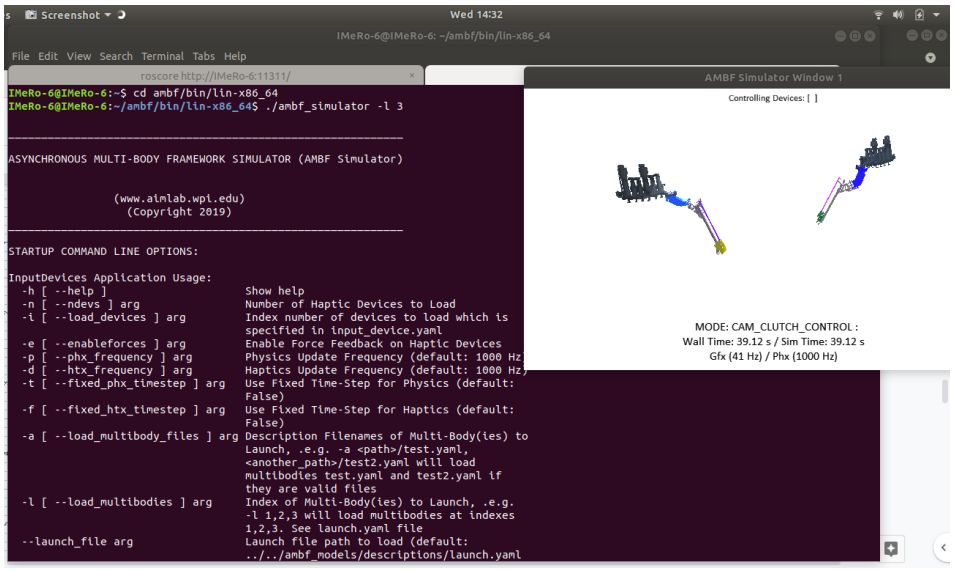
5. Run `roscore`



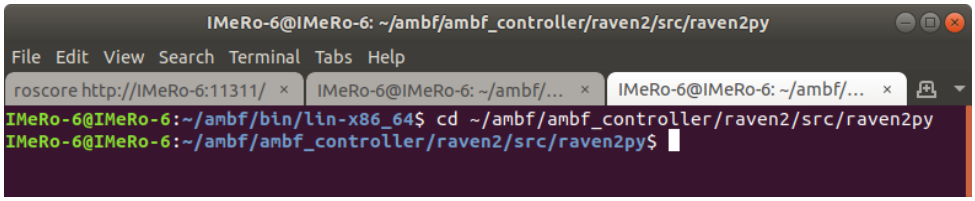
6. Open a new tab and navigate to /ambf/bin/<os>



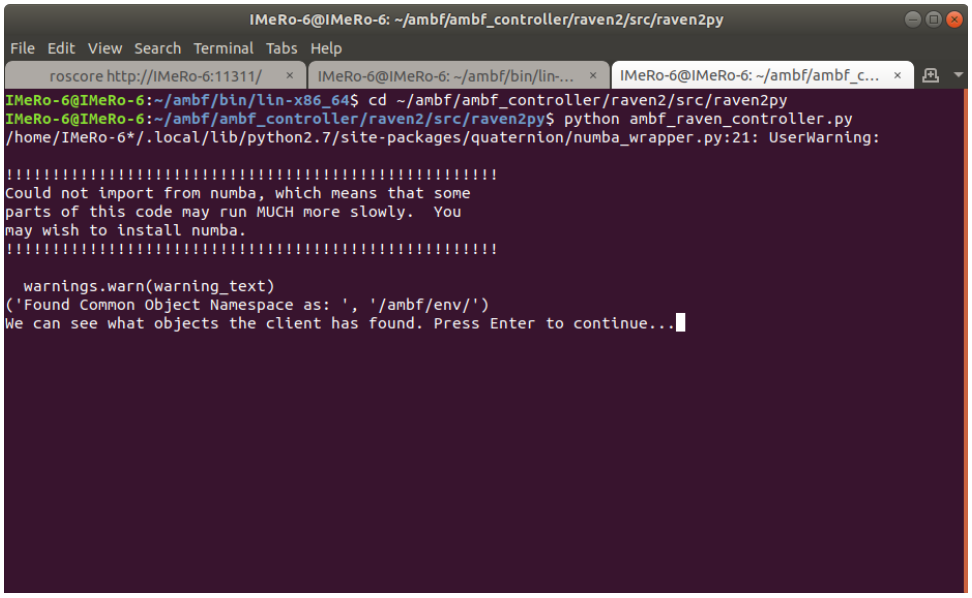
7. Run `./ambf_simulator -l 3`



8. Open a new tab, navigate to `~/ambf/ambf_controller/raven2/src/raven2py`



9. Run `python ambf_raven_controller.py` and follow instructions in the terminal



Pieces of the Controller

ambf_raven_def.py

This file defines important constants for mapping the motion trajectory of the Raven II, whether that be homing, sine dance, or manual control, adapted from the file ambf_defines.cpp in the AMBF original git repository. It also contains a lot of other information about the measurements and characteristics of the Raven II, including:

CAMERA_COUNT	RAVEN_JOINTS	RAVEN_ARMS
RAVEN_IKSOLS	LOOP_RATE	Eps
MAX_JOINTS	MIN_JOINTS	HOME_JOINT
DANCE_SCALE_JOINTS	RAVEN_JOINT_LIMITS	RAVEN_DH_ALPHA

RAVEN_DH_THETA	RAVEN_DH_A	RAVEN_DH_D
RAVEN_IKIN_PARAM	RAVEN_T_B0	RAVEN_T_CB

ambf_raven.py

This file defines the ambf_raven object which contains attributes and methods for controlling the simulated raven. It connects the controller to the Raven II in the simulator upon initialization and uses values from the ambf_raven_def.py to help plan motion for the various control modes.

Variables

_client	homed	moved
limited	arms	start_jp
delta_jp	home_joints	next_jp
dance_scale_joints	loop_rate	raven_joints
rc	rampup_count	i
speed	rampup_speed	

Methods

go_home	sine_dance	manual_move
move	control_move	

go_home

This method gradualizes the movement of the Raven II arms from the starting position to the home position, defined using the `home_joints` array. The method should be called `loop_rate` times and should reach the final position upon the last call of the method. If the arm is homed by the end of the final iteration, the method should return True. Otherwise it returns False. The application of this method will make more sense in the context of the `ambf_raven_controller.py`.

`sine_dance`

This method slowly moves the Raven arm in a sine motion, using a sine function to move each joint. Similar to the method above, the function should be repeatedly called in a loop until the user changes control modes.

`manual_move`

This used forward and inverse kinematics to create a new desired joint position array based on the change in cartesian position. In planning this method, I basically following this model:



`move`

This method uses a similar process to the `go_home` method, but instead of `HOME_JOINTS`, the method will gradually move the arm to the next desired position passed into the function. Because of the nature of the process, the movement will always take the same speed, regardless of the distance needed to travel.

`control_move`

This method isn't used in the final controller, but it was intended for use with key inputs.

raven_fk.py

This file contains all the methods for performing forward kinematics. There is more information in the documentation of the file for the specific processes for deriving the transformation matrices from the joint positions.

raven_ik.py

This file contains all the methods for performing inverse kinematics. There is more information in the documentation of the file for the specific processes for deriving the joint positions from the transformation matrix.

ambf_raven_controller.py

This file is what I would consider the main function of the control program. It is the one that controls the user inputs and “talks” to the raven, for it to move. It uses an array of control switches (an array of booleans) to turn on and off certain control modes: home, sine dance, manual control and quit. In order to simultaneously wait for user input and also operate the robot, the file uses python threading and queues to communicate between the threads. For more information, look at the documentation for the file.

utilities.py

This file already existed in the ambf python controller for dVRK in the original github repository, however I copied it for my controller and added a few functions to mimic the methods defined in the C++ class for transformation matrices.

Capabilities of the Controller

Homing

When the homing control mode is entered, both raven arms move to a predefined position defined in the ambf_raven_def.py file. Once it reaches that position, it will stay until another control mode is entered.

Sine Dance

When the sine dance mode is entered, the raven arms will move in a dance pattern defined by a sine function in the ambf_raven.py file. It will continue to move until another control mode is entered.

Manual Move

When the manual mode is entered, the raven arms will move in a predefined direction based on hard-coded instructions in the file. This can be hard coded by setting the values in the arrays for each x, y, z array to the distance in each direction each arm should move. It will move until that position has been assumed and then stay in that position.

Limitations of the Controller

Slow to move short distances

The raven will take the same amount of time to move any distance, therefore to travel a small distance, it will take the same amount of time as moving a further distance.

No manual controller in real time

As of now, the controller does not allow control in real time. This is being worked on.

Breaks when it makes contact with soft bodies

Next steps

Notes from Adnan talk

1. Using ambf 2.0 for using soft body, still actively working on upgrading it, finishing last few things, all users can use plug-in (external software that any of us can write that customizes how any bodies in ambf to interact with each other).
    a. Ex. create simulation with path planning for knee surgery, draw on the knee where they want to go, all customized with additional software. Goal to set up interface that can take any of the plug-ins
    b. Planning on releasing official ambf 2.0 within a few months, fully ready to go.
    c. Might make sense to wait a little
2. Raven exploding when touching soft bodies, using this software called Blendr, creating 3D models, just building the model. Made all of the soft bodies in their, for dvrk he pulled that model back into Blendr. Problem with raven, each of the joints and links are weirdly shaped, define collision by the shape of each of the links.
    a. Concave → makes ambf extremely hard to calculate collision.
    b. For dVRK, showed on Blendr there is a feature where you can define collision shape. Created cylinder blocks that the smallest cylinder block that is outside

the original shape of link to simplify collision calculation (also other robots, gripper)

c. Normal that Raven will explode. Quick solution → use another robot. Eventually want raven support → download Blendr and use it.

3. Don't need ambf just yet, so easy to just load raven directly into blendr, change squishability, etc. collect data
4. Two directions
    a. Make whatever we have compatible with 2.0, push to Adnan's official stuff.
    b. Aim for IRC → rosbag, create ML model to predict actual raven torque based on ambf torque.
        i. Mathematically possible, but there is a lot of noise (friction, etc.)

New branch → raven_ml

UPDATE: 28th July 2021

New elements:

"Manual mode"

In the devel_sean_r branch, there is another control mode called "manual" mode which uses inverse and forward kinematics to move the robot tip in cartesian space. By hard coding in the desired x, y, z point, the robot can move slowly to the desired position.

New Branch → raven_ml

I created a new branch called "raven_ml" specifically for collecting data and processing it for machine learning. It features a new control mode called "file" mode which allows the user to input a .csv file of the joint positions for the actual raven (num timestamps x num joints matrix). With this new mode, one can record the joint positions from a real raven and then have the simulated raven copy that movement. This allows for the collection of ambf data that could be aligned to data from the physical raven.

Progress on collecting data

I created two python scripts in order to organize the data. The first script takes data from different rostopics (correlated to different joints) and combines them, aligning by timestamp, by comparing the time stamps for the baselink rows and looking for the closest matching row for each other joint. This creates a larger dataset containing information about joint positions, joint velocities and twist for each joint.

The second script is supposed to combine physical raven and ambf raven data based on joint position, however it is in prototype because we did not implement a successful method to finding similar rows. This will be a project for the Fall 2021.

Next Steps

Because of the random timestamps for the two different ravens, we had some trouble successfully sorting and finding the corresponding rows of data simply with jpos and time data. As a next step, it makes sense to fit a sine curve to our data and then use that to align a beginning and ending to the movement. Ideally this method would allow more effective and accurate data wrangling with which we could train our model.

